

Lab1 实验报告

姓名: 郑依桐
学号: 25300180083

一、规则检查截图

./check_ops.py bits.c 运行截图:

```
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 11/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 11/16 operators
roundEvenPow2: 22/24 operators
midpointTowardFirst: 22/32 operators
isBetweenEitherOrder: 47/48 operators
mul5Sat: 30/30 operators
classifyAdd3: 41/52 operators
floatScaleThreeHalves: 35/60 operators
floatRoundEven: 25/65 operators
float_i2f: 30/40 operators
bitCount: 24/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

./btest 截图:

Score	Rating	Errors	Function
1	1	0	signMask
2	2	0	bitXor
3	3	0	negativePart
4	4	0	copyByteWithin
4	4	0	logicalShift
4	4	0	swapNibblePairs
4	4	0	secondLowestZeroBit
5	5	0	oddParity
5	5	0	rotateRightBits
5	5	0	roundEvenPow2
5	5	0	midpointTowardFirst
7	7	0	isBetweenEitherOrder
7	7	0	mul5Sat
7	7	0	classifyAdd3
7	7	0	floatScaleThreeHalves
10	10	0	floatRoundEven
10	10	0	float_i2f
10	10	0	bitCount
10	10	0	bitReverse
Total points: 110/110			

二、 实现思路

本次实验我严格遵循了整数题和浮点题的编码规则，实现了全部 19 个函数。以下是各个函数的实现思路简述：

P1 signMask:

要求：表示 0x80000000

逻辑：利用左移操作，将 1 左移 31 位，直接得到 0x80000000。

P2 bitXor:

要求：异或运算

逻辑：异或即 $\sim(\text{两者都为 } 1) \ \& \ \sim(\text{两者都为 } 0)$

P3 negativePart:

要求：负数返回绝对值，正数返回 0

逻辑：

1. $x >> 31$ 得到符号位：正数 0x80000000（即 0），负数 0xFFFFFFFF（即 -1）
2. $\sim x + 1$ 取相反数：如果 x 是正数：sign=0，&运算得到 0；如果 x 是负数：sign 是 -1，& 运算等于 $\sim x + 1$ 本身

P4 copyByteWithin:

要求：将整数 x 中的第 src 个字节，拷贝到第 dst 个字节的位置

逻辑：

1. 计算源字节需要右移的位数与目标字节需要左移的位数
2. 0xFF 用来提取或清除一个字节的的数据
3. x 右移，把源字节移到最低 8 位
4. 把 x 中目标字节的位置清零
5. 用 | (按位或) 把两部分拼起来

P5 logicalShift:

要求：逻辑右移

逻辑：

1. 算术右移
2. 构造一个掩码，用来清除掉刚才多补的那些 1
3. & 运算，把算术右移多补的 1 全部清除为 0

P6 swapNibblePairs:

要求：交换 x 中每个字节内部的高 4 位和低 4 位

逻辑：

1. 构造掩码 0x0F0F0F0F
2. 提取出所有字节的低 4 位，高 4 位清零；将低 4 位左移 4 位，放到高 4 位的位置；再合并二者

P7 secondLowestZeroBit :

要求：返回一个掩码，只有 x 中第二低的 0 位是 1

逻辑：

- 1.把 x 取反，题目变成找 z 中第二低的 1 位
- 2.只保留 z 中最右边的那一个 1，其余位全部置 0，删掉这个 1
- 3.在剩余的位中，再次提取最低的 1 位

P8 oddParity:

要求：计算 x 二进制中 1 的个数，偶数返回 1，奇数返回 0

逻辑：

- 1.将 x 的高 16 位和低 16 位进行异或，1 的个数的奇偶性保持不变
- 2.一步步折叠至只剩一位，提取最低位，如果 1 的总数是奇数，第 0 位是 1；如果是偶数，第 0 位是 0

P9 rotateRightBits:

要求：循环右移

逻辑：

- 1.取模到 0~31
- 2.计算绕回来的部分
- 3.算术右移，用掩码高位补 0

P10 roundEvenPow2:

要求：将非负整数 x 舍入到最接近的 2^n 的倍数

逻辑：

- 1.构造掩码：低 n 位为 1，其余为 0
- 2.位运算计算半值
- 3.分离基数和余数
- 4.判断余数是否大于等于半值
- 5.提取商
- 6.判断 s 是否为非负数，是否为 0
- 7.提取商的最低位，判断是否向上舍入，最终计算

P11 midpointTowardFirst:

要求：返回 $(x + y)/2$ 的精确数学中点，但不能产生溢出，如果精确中点在两个整数正中间，则选择靠近第一个参数 x 的那个整数

逻辑：

- 1.分解加法防止溢出
- 2.判断是否需要进 1：如果 b 是负数，说明 x 和 y 异号
- 3.同号且 $x > y$ 时，需要向上调整；异号且 x 为非负数时，需要向上调整；只有在需要调整且中点是奇数时，才加 1

P12 isBetweenEitherOrder:

要求：不使用任何比较运算符和减法，判断一个数 x 是否在 a 和 b 之间（包含边界）

逻辑:

- 1.提取符号位
- 2.计算符号位异或, 判断两个数是否异号
- 3.计算差值, 判断差值是否非负
- 4.提取 x, a, b 是否为非负数
- 5.判断 x 是否在 a, b 之间: 如果同号, 直接看 `same1`; 如果异号, 看 `nxx` 或 `nax`
- 6.组合条件满足一个就返回 1: x 在 a 上方且 b 在 x 上方; x 在 b 上方且 a 在 x 上方

P13 mul5Sat:

要求: 计算 $x*5$; 如果结果溢出, 根据溢出的方向返回 `INT_MAX` 或 `INT_MIN`

逻辑:

- 1.定义溢出边界 `T`
- 2.提取 x 的符号位
- 3.计算 x 的正负掩码
- 4.计算 $x*5$
- 5.检测正、负溢出
- 6.构造饱和值、根据溢出标志选择饱和值

P14 classifyAdd3:

要求: 判断三个整数相加 $x+y+z$ 是否发生了溢出, 并给出溢出方向

逻辑:

- 1.执行两次加法
- 2.提取所有相关数的符号位
- 3.提取是否为负、是否为非负
- 4.检测两次加法的溢出情况
- 5.合并两次溢出的净值, 决定返回值

P15 floatScaleThreeHalves:

要求: 单精度浮点数乘以 1.5

逻辑:

- 1.拆解 32 位编码
- 2.处理 NaN 和无穷大
- 3.处理 $\text{exp}=0$ 的数
- 4.处理 $\text{exp} \neq 0$ 的数
- 5.归一化 P , 处理舍入
- 6.处理舍入后的再次溢出, 计算最终值

P16 floatRoundEven:

要求: 将一个浮点数舍入到最接近的整数, 并返回该整数的浮点数表示

逻辑:

- 1.拆解 32 位编码
- 2.处理 NaN 和无穷大
- 3.处理 $\text{exp}=0$ 的数
- 4.计算真实的指数 $e = \text{exp} - 127$

- 5.处理已经是整数的情况
- 6.处理指数为负数的情况
- 7.处理一般情况
- 8.四舍六入五成双
- 9.处理舍入后为 0 的情况
- 10.将舍入后的整数重新转换为浮点数格式：找到最高有效位，并据此确定新的指数
- 11.拼装最终的浮点数表示

P17 float_i2f:

要求：将一个 32 位有符号整数转换成单精度浮点数的位级表示

逻辑：

- 1.处理特殊情况：0
- 2.提取符号并计算绝对值
- 3.寻找最高有效位的位置 n
- 4.计算阶码
- 5.处理尾数和舍入，以及舍入导致的溢出
- 6.拼装最终的 32 位浮点数表示

P18 bitCount:

要求：计算一个 32 位整数中 1 的个数。

逻辑：

- 1.定义基础掩码，分别用于提取奇数位、每 4 位中的低 2 位、每 8 位中的低 4 位
- 2.将基础掩码扩展到 32 位
- 3.每一步都把相邻的组相加，按步计算每 2、4、8、16、32 位中 1 的个数

P19 bitReverse:

要求：反转整数 x

逻辑：

- 1.定义基础掩码，分别提取奇数位、每 4 位中的低 2 位、每 8 位中的低 4 位、每 16 位中的低 8 位、每 32 位中的低 16 位
- 2.逐步交换相邻的 1、2、4、8、16 位

三、实验感受、对助教的建议

本次实验可以算是本课程真正的入门章节了，在刚开始写这次的 lab 时，我对仓库、VS Code 等等的理解还处在一个非常浅层的状态，因此在理解作业要求、配置环境的过程中遇到了非常多的障碍，可以说是每一步都在报错，每一行要求都需要费力地查资料理解，到最后完成这次 lab 的时候，我对整个 Git 模式的感受都有了非常大的变化。解决这 19 道题是一个熟能生巧的过程，花了大量时间耐心理解后，就能感受到题目对于我们计算机底层补码表示、位运算等内容最深入本质的考察。可以说这是目前我接触到的所有课程任务中最能让我感受到计算机底层逻辑的一次，十分感谢教授和助教们。有一点建议是，可以在环境配置文档中，补充说明一下 check_ops.py 等脚本在 Windows 或者 MacOS 下可能存在的兼容性问题及解决方案。